

1. DDL COMMANDS WITH CONSTRAINTS

AIM: To execute DDL commands CREATE, ALTER, RENAME, TRUNCATE and DROP with constraints.

ALGORITHM:

Step 1: Start Oracle SQL.

Step 2: Create table STUDENT and check with DESC.

Step 3: Create table STU with PRIMARY KEY, NOT NULL, UNIQUE and CHECK.

Step 4: Use ALTER to drop the SCORE column, add GENDER and modify AGE.

Step 5: Rename STUDENT as STU10 and column EMAIL as EMAILID.

Step 6: Use TRUNCATE to delete all records of STU.

Step 7: Use DROP to remove STU and check with DESC.

Step 8: Stop the execution.

RESULT: Thus, the program was executed successfully and the output was verified.

2. DML COMMANDS WITH CONSTRAINTS

AIM: To perform DML commands INSERT, SELECT, UPDATE and DELETE on a table.

ALGORITHM:

Step 1: Start Oracle SQL.

Step 2: Use the STU10 table (stid, stname, gender).

Step 3: INSERT five student records.

Step 4: SELECT all records, only names, and only female students (WHERE).

Step 5: UPDATE the student name, then name and gender.

Step 6: DELETE one record and display the table.

Step 7: DELETE all records and display the table.

Step 8: Stop the execution.

RESULT: Thus, the program was executed successfully and the output was verified.

3. QUERIES, SUBQUERIES & AGGREGATE FUNCTIONS

AIM: To execute basic queries, aggregate functions, subqueries and join queries.

ALGORITHM:

Step 1: Create table EMPL and insert employee records.

Step 2: Display records using SELECT and WHERE.

Step 3: Use COUNT, SUM, AVG, MAX and MIN.

Step 4: Use ORDER BY and GROUP BY.

Step 5: Use a subquery to find salary above the average.

Step 6: Create table DEPT and insert department records.

Step 7: Find department-wise employee count using a subquery with JOIN.

Step 8: Perform INNER, LEFT, RIGHT and CROSS JOIN.

Step 9: Stop the execution.

RESULT: Thus, the program was executed successfully and the output was verified.

4. PL/SQL – EXCEPTION HANDLING

AIM: To handle Predefined, User-Defined, Named System and OTHERS exceptions in PL/SQL.

ALGORITHM:

Step 1: Start the program and declare the variables.

Step 2: Divide a number by zero and handle ZERO_DIVIDE.

Step 3: Raise the user-defined exception insufficient_balance if the balance is low.

Step 4: Select a missing employee and handle NO_DATA_FOUND.

Step 5: Run a wrong dynamic SQL and handle it using OTHERS.

Step 6: Display the messages.

Step 7: Stop the execution.

RESULT: Thus, the program was executed successfully and the output was verified.

5. PL/SQL – CURSOR

AIM: To use implicit, explicit, cursor FOR loop and parameterized cursors in PL/SQL.

ALGORITHM:

Step 1: Start the program and display the EMP table.

Step 2: Implicit cursor: update a salary and check SQL%ROWCOUNT.

Step 3: Explicit cursor: declare, open, fetch in a loop, display and close.

Step 4: Cursor FOR loop: display records without OPEN, FETCH and CLOSE.

Step 5: Parameterized cursor: display employees of a given department.

Step 6: Stop the execution.

RESULT: Thus, the program was executed successfully and the output was verified.

6. PL/SQL – TRIGGER

AIM: To create and execute triggers in PL/SQL.

ALGORITHM:

Step 1: Create table students (roll_no, name, marks).

Step 2: BEFORE INSERT trigger: set marks to 0 if null.

Step 3: AFTER INSERT trigger: display a message.

Step 4: Insert a record and display the table.

Step 5: BEFORE UPDATE trigger: limit marks to 100.

Step 6: Update marks to 120 and display the table.

Step 7: AFTER DELETE trigger: display a message, then delete the record.

Step 8: Stop the execution.

RESULT: Thus, the program was executed successfully and the output was verified.

7. PL/SQL – PACKAGES

AIM: To create a PL/SQL package with a function and use a predefined package.

ALGORITHM:

Step 1: Start the program.

Step 2: Create package specification math_pkg with function square_num.

Step 3: Create the package body to return the square of a number.

Step 4: Call the function with the value 5 and display the result.

Step 5: Use DBMS_RANDOM to display a random number and a random string.

Step 6: Stop the execution.

RESULT: Thus, the program was executed successfully and the output was verified.

8. LIBRARY MANAGEMENT SYSTEM

Experiment: Visual Basic 2010 and Oracle 10g Integration – Library Management System

AIM: To develop a Library Management System using Visual Basic 2010 and Oracle 10g to add, issue, return and display book records.

ALGORITHM:

- Step 1:** Start Visual Basic 2010 and create a new Windows Form.
- Step 2:** Add textboxes for Book ID, Book Name, Author Name and Quantity.
- Step 3:** Add buttons Add Book, Issue Book, Return Book and View, and a DataGridView to show the records.
- Step 4:** Create the book table in Oracle 10g with Book ID, Book Name, Author Name and Quantity.
- Step 5:** Write the connection string in the code to connect VB to Oracle 10g.
- Step 6:** Add Book: enter the book details and insert them into the table.
- Step 7:** Issue Book: enter the Book ID and decrease the quantity by 1.
- Step 8:** Return Book: enter the Book ID and increase the quantity by 1.
- Step 9:** View: retrieve all book records and display them in the DataGridView.
- Step 10:** Display a success or error message for each operation.
- Step 11:** Stop the execution.

RESULT: Thus, the program was executed successfully and the output was verified.

9. STUDENT MARKSHEET PROCESSING

Experiment: Visual Basic 2010 and Oracle 10g Integration – Student Marksheet Processing

AIM: To develop a Student Marksheet Processing application using Visual Basic 2010 and Oracle 10g to calculate total, result and percentage.

ALGORITHM:

Step 1: Start Visual Basic 2010 and create a new Windows Form.

Step 2: Add textboxes for Student ID, Name, four subject marks (OS, RDBMS, CN, SE), Total, Result and Percent.

Step 3: Add buttons Calculate, Insert, Delete and View, and a DataGridView to show the records.

Step 4: Create the ST table in Oracle 10g with student details, marks, total, result and percent.

Step 5: Write the connection string in the code to connect VB to Oracle 10g.

Step 6: Calculate: $\text{total} = \text{sum of the four marks}$ and $\text{percent} = \text{total} / 4$.

Step 7: If any mark is 30 or below, the result is FAIL, otherwise PASS.

Step 8: Insert: save the student record in the table.

Step 9: Delete: enter the Student ID and remove that record.

Step 10: View: retrieve all records and display them in the DataGridView.

Step 11: Display a success or error message for each operation.

Step 12: Stop the execution.

RESULT: Thus, the program was executed successfully and the output was verified.

10. PAYROLL PROCESSING

Experiment: Visual Basic 2010 and Oracle 10g Integration – Payroll Processing

AIM: To develop a Payroll Processing application using Visual Basic 2010 and Oracle 10g to calculate gross and net salary.

ALGORITHM:

Step 1: Start Visual Basic 2010 and create a new Windows Form.

Step 2: Add textboxes for Employee ID, Name, Basic Salary, Allowance, Deduction, Gross and Net.

Step 3: Add buttons Calculate, Insert, Delete, View and Exit, and a DataGridView to show the records.

Step 4: Create the EMP table in Oracle 10g with employee ID, name, salary, gross and net.

Step 5: Write the connection string in the code to connect VB to Oracle 10g.

Step 6: Calculate: allowance = 25% of basic and deduction = 12% of basic.

Step 7: Gross = basic + allowance and Net = gross – deduction.

Step 8: Insert: save the employee record in the table.

Step 9: Delete: enter the Employee ID and remove that record.

Step 10: View: retrieve all records and display them in the DataGridView.

Step 11: Exit: close the application.

Step 12: Stop the execution.

RESULT: Thus, the program was executed successfully and the output was verified.